next-workflow 2.3

A step-by-step guide to adding this Claude Code workflow to your Next.js app, and what it does once it's in.

1. What is next-workflow?
2. What you need first
3. Installing it, step by step
4. Your first task: plan, approve, build
5. Running many tasks safely
6. All commands and helpers
7. The specialist agents
8. Automatic checks (hooks)
9. The coding rules it follows
10. Settings you can change
11. A typical day
12. Problems and fixes
13. What's new in 2.3

1. What is next-workflow?

next-workflow is a **kit for Claude Code**, not a Next.js library. Nothing is added to your app's code or bundle. It copies instructions and checks into your project's `.claude/` folder, and they change how Claude Code works on your Next.js project.

- **It plans before it builds.** Opus turns your request into small task files with clear "done" checks. You read and approve them.
- **Sonnet builds each task.** It works directly for small tasks and uses specialist agents for large ones.
- **A second agent checks the work.** It sees only the task and the changes, never the builder's reasoning.
- **Automatic checks run on every edit** and catch Next.js mistakes before the build does.
- **It remembers your project** in `.claude/project-context.md`, so it never re-learns it.
- **It's safe to undo.** You can get one commit per task and roll back any single task.
- **It measures itself:** time, cost and check hits per task.

> **In one sentence**
> Describe what you want with `/next:write-task`, approve the plan, run `/next:task`, and every task is built, independently verified and recorded.

2. What you need first

Requirement	Why
A Next.js 15 or 16 project	Its `package.json` must list `"next"`. The workflow detects the version and follows the right rules.
Claude Code	The workflow runs inside it (terminal, desktop app or IDE).
bash, Node.js	The checks are shell scripts (Linux, macOS, or WSL on Windows).

`jq`	Needed for safe settings merging and for batch runs.
git (recommended)	Enables task diffs, the verifier's view of changes, per-task commits and undo.
Playwright (optional)	Lets the verifier check `[browser]` criteria in a real browser.

3. Installing it, step by step

1 Install into your app

```
cd ~/www/hobbies/next-workflow
./install.sh /path/to/your-next-app           # add --dry-run to preview
```

This copies the kit into `your-next-app/.claude/`, merges `settings.json` (it backs up the old one and replaces any older kit entries, so nothing is duplicated), stamps the kit version, and records any type errors that already exist so Claude is only held to errors it introduces.

2 Open Claude Code in your app

```
cd /path/to/your-next-app
claude
```

You'll see a short report: kit version, Next version, Tailwind, test tools and verify commands. That means it's working.

3 Run setup once

```
/next:setup
```

It creates `.claude/tasks/` (where your task files live), fills `.claude/project-context.md` with facts about your repo (folders, import alias, auth helper), adds a short pointer to `CLAUDE.md`, and tells you what won't work here (for example "no git, so no undo").

> **If setup says the typecheck is already failing**
> Type `! bash .claude/hooks/verify.sh --baseline` in the prompt. That records the existing errors. Claude isn't allowed to do this itself, so it can never excuse its own mistakes.

> **Commit**
> `.claude/` (which includes `.claude/tasks/`) and `CLAUDE.md` so your team gets the same workflow. `.claude/.cache/` is already git-ignored.

4. Your first task: plan, approve, build

1 Describe what you want

```
/next:write-task a projects page where users see their projects and can archive
one
```

The planner (Opus) asks questions first if something is unclear, splits big requests into small tasks, sets the order (`depends`) and size, and writes **drafts**. It never writes code.

2 Read the drafts, then approve

```
---
id: 003
title: Projects page with archive
status: draft
depends: 002
size: large
---
## Goal
Signed-in users see their own projects at /projects and can archive one.

## Acceptance
- [ ] [review]  /projects lists only the signed-in user's projects
- [ ] [test]    Archiving someone else's project fails and changes nothing
- [ ] [browser] Signed-out users are redirected to /login

## Notes
Out of scope: un-archive.
```

Each "done" check carries a tag saying how it will be proven:

- `[test]`: a test must prove it.
- `[browser]`: it has to be seen in the running app.
- `[review]`: reading the code is enough.

Edit anything you disagree with, then:

```
bash .claude/hooks/tasks.sh approve all        # or: approve 003
```

3 Build it

```
/next:task
```

Sonnet takes the next approved task and works through it:

1. It builds the task: directly if small, with the full phased build if large.
2. It runs the checks.
3. It hands the task and its diff to the **verifier**.
4. If any criterion fails, it fixes it and verifies again (two rounds at most, then it marks the task blocked).
5. It closes the task with a log line and saves what it learned.

5. Running many tasks safely

Running tasks one after another in a single conversation lets context pile up, and quality drops by the 4th or 5th task. So `/next:task all` stops after 3. For longer batches use the runner, which starts a **fresh session for every task**:

```
bash .claude/hooks/run-tasks.sh --checkpoint          # all approved tasks, in order
bash .claude/hooks/run-tasks.sh 004 005 --budget 3    # just these, max $3 each
```

Option	Effect
`--checkpoint`	One local git commit per finished task, named `task 004: …`, on a new branch `nw/batch-<time>` so `main` is never touched. Each commit holds only that task's files: never secret files, never edits that were already there. Nothing is pushed.
`--no-branch`	With `--checkpoint`, commit on the current branch instead
`--budget USD`	Spending limit per task
`--max N`	Stop after N tasks
`--dry-run`	Show what would run, and run nothing

The runner stops at the first task that doesn't finish (blocked, failed or over budget). Run it again and it continues from there. It prints time, turns and cost per task.

Undo one task (needs `--checkpoint`): `bash .claude/hooks/tasks.sh undo 004`. That task's code is reverted and it goes back to `todo`. If later finished tasks depend on it, undo refuses and lists them. `undo 004 --cascade` undoes those first, newest first.

When the batch looks good: `git switch main && git merge nw/batch-…` (the runner prints the exact command).

draft → you approve → **todo** → **doing** → **done** or **blocked**

6. All commands and helpers

Slash command	Model	Use it to…
`/next:setup`	current	Connect the workflow to the project. Run once.
`/next:write-task <what>`	Opus	Plan work into draft task files. Never writes code.
`/next:task [id|all|list]`	Sonnet	Build tasks: your main command.
`/next:feature <id>`	Sonnet	Force the full phased build (data → page → UI → tests → review) on a task.
`/next:test [target]`	current	Write or fix tests outside a task.
`/next:review [target]`	Opus reviewer	Review your changes for bugs and security holes. It doesn't change anything.
`/next:audit [path]`	current	Performance, accessibility and correctness audit, ranked.
`/next:ship`	current	Typecheck, lint, tests and build, fixing what fails. Never commits.

Terminal helper	Does
`bash .claude/hooks/tasks.sh list`	Status of every task
`… tasks.sh approve <id|all>`	Approve drafts
`… tasks.sh diff <id>`	Everything a task changed
`… tasks.sh undo <id> [--cascade]`	Revert a task's checkpoint commit (and its dependents with `--cascade`)
`… tasks.sh map`	A short map of the app (routes, components, lib modules), cached
`… tasks.sh context <id>`	The task plus what its dependencies did and changed
`… tasks.sh dir`	Which folder holds the tasks (normally `.claude/tasks`)
`… tasks.sh migrate`	Move an older project's `tasks/` folder to `.claude/tasks/`
`… tasks.sh stats`	Tasks done, time per size, batch cost, **verifier first-pass rate**, edit-hook speed, which checks fire most
`bash .claude/hooks/run-tasks.sh`	Batch runner, with a fresh session per task

7. The specialist agents

Agent	Job	Can edit?
`next-route-builder`	Pages, layouts, loading, error and not-found files, metadata	Yes
`next-ui-builder`	Components, styling, accessible markup	Yes

`next-data-layer`	Server Actions, queries, caching, API routes	Yes
`next-test-writer`	Vitest and Playwright tests	Yes
`next-verifier` (new)	Checks every "done" criterion against the task and diff only (Sonnet)	No
`next-reviewer`	Security and correctness review (always Opus)	No
`next-perf-auditor`	Bundle size, waterfalls, images and fonts	No
`next-a11y-auditor`	Keyboard, focus, labels, contrast (no shell access at all)	No

8. Automatic checks (hooks)

Check	When	What it does
`session-context`	Session start	Reports the kit version and your stack, points to the Next 15 or 16 rules, and loads the project memory.
`post-edit`	After each edit	One entry point that runs every edit check: App Router mistakes, component rules, Tailwind classes, then formatting. It gives one combined answer.
`bash-guard`	After shell commands	Files written through the shell (not Claude's edit tool) get the same checks, and still trigger the typecheck.
`guard-secrets`	Before shell commands	Blocks commands that read `.env` files or private keys. `.env.example` is allowed.
`verify`	Before Claude finishes	Typecheck. Only **new** errors fail; errors that already existed are baselined.

Every check hit is recorded in `.claude/.cache/metrics.jsonl`. `tasks.sh stats` summarises them.

9. The coding rules it follows

- **Server Components by default.** Interactivity goes in small separate "island" files (`*.client.tsx`).
- **Components only render.** Logic goes into hooks, services, helpers and schemas.
- **zod** at every input boundary. **zustand** only when needed, created per request.
- **Server Actions are public endpoints:** authenticate, authorize, validate, then change data.
- **Version-aware:** Tailwind v4 classes for the version you actually have installed. On Next 16 it uses `proxy.ts`, `'use cache'` / `cacheLife` / `cacheTag` and `updateTag`.
- **Accessibility is part of the job**, and your existing style always wins.

10. Settings you can change

Set these before starting `claude` (or before the batch runner):

Variable	Values	Effect
`NEXT_WF_CHECKPOINT`	`commit`	One local commit per finished task (the runner's `--checkpoint` sets this)
`NEXT_WF_DEV_URL`	e.g. `http://localhost:3000`	Lets the verifier check `[browser]` criteria in a real browser (needs `@playwright/test` in the project; the kit brings its own Playwright config)
`NEXT_WF_VERIFY`	`typecheck`, `typecheck,lint`, `off`	What runs before Claude finishes
`NEXT_WF_COMPONENT_GUARD`	`enforce`, `warn`, `off`	Component rules: block, only advise, or off
`NEXT_WF_STYLE_GUARD`	`on`, `off`	Tailwind check
`NEXT_WF_SECRETS_GUARD`	`on`, `off`	Blocking of `.env`/key reads
`NEXT_WF_FORMAT`	`all`, `prettier`	`prettier` skips the per-edit ESLint fix (faster on big repos)
`NEXT_WF_METRICS`	`on`, `off`	Recording to `metrics.jsonl`

Update: run `./install.sh /path/to/app` again. **Remove:** `./install.sh /path/to/app --uninstall`. That removes the kit's files and hook entries and keeps your tasks, memory and own settings.

11. A typical day

1. `/next:write-task …`, then answer its questions.
2. Read the drafts in `.claude/tasks/`, tweak them, then `tasks.sh approve all`.
3. Optional: start `npm run dev` and set `NEXT_WF_DEV_URL` so browser checks really run.
4. `bash .claude/hooks/run-tasks.sh --checkpoint` and let it work.
5. Check the result. Undo any task you don't like with `tasks.sh undo <id>`.
6. `/next:review`, then `/next:ship`. When everything is green, push it yourself.
7. Now and then, run `tasks.sh stats` to see cost, time and which checks fire most.

12. Problems and fixes

Problem	Fix
Every turn fails on old type errors	Record them once: `! bash .claude/hooks/verify.sh --baseline`
"task is still a draft"	Read it, then `tasks.sh approve <id>`.
A task is `blocked`	Read the reason in its Log section (in headless runs it's the question Claude would have asked), add the answer to Notes, set it back to `todo`, and run it again.

The verifier marks `[browser]` items UNVERIFIED	Start the dev server and set `NEXT_WF_DEV_URL`, with Playwright installed.
`undo` says there's no checkpoint	The task was closed without `--checkpoint`. Use `tasks.sh diff` and revert by hand.
"checkpoint: SKIPPED — secret files changed"	A `.env` or key file changed during the task. Add it to `.gitignore`, then commit the task's files yourself. Nothing was committed.
"project-context.md … only the first 4000 were loaded"	The memory file is too long: merge and prune it to under ~60 lines.
Claude asks permission to edit a file in `.claude/`	Normal: Claude Code protects that folder. New drafts don't ask, because they're written by `tasks.sh new`. Approve edits you asked for, or pick "allow for this session".
A note in `.claude/tasks/` isn't listed as a task	Intended: only files whose first line is `---` are tasks.
Your tasks are still in `tasks/` after an upgrade	They keep working. Move them with `bash .claude/hooks/tasks.sh migrate`.
A command was blocked for touching `.env`	That's intended. Run it yourself, or use `.env.example`.
A check blocks something you want	Set that check to `warn` or `off` (section 10).
Installer: no "next" dependency	Point it at the folder that holds your app's `package.json`.

13. What's new in 2.3

Where your tasks live now: `.claude/tasks/`

Task files are kept inside `.claude/tasks/`, next to the rest of the kit's project state, instead of a `tasks/` folder in the project root. `/next:setup` creates it.

- **Only real tasks count.** A file is a task only if its first line is `---`. Notes, plans and other `.md` files in the folder are ignored and never run. A `tasks/` folder of your own notes in the project root is left alone.
- **No permission prompt per draft.** Claude Code asks you to approve every file Claude writes inside `.claude/`, and settings can't turn that off. So the planner saves drafts with `tasks.sh new`, which writes them without a prompt. It accepts only a properly formed **draft** and can't create a task that is ready to run. When you ask Claude to change a draft afterwards, you approve that one edit as usual.
- **Upgrading an older project** that has a `tasks/` folder made by the kit: it keeps working. When you're ready, run `bash .claude/hooks/tasks.sh migrate` to move it. The installer reminds you.

Remembering context, to use fewer tokens

Feature	What it saves
`tasks.sh map`	A short, cached map of the app, read instead of exploring with many searches. It is rebuilt only when files change.
`tasks.sh context <id>`	The task, plus what each task it depends on did and which files it changed, so code that was just written isn't read again.
Subagent context	Every kit agent starts with the stack report and project memory, about 550 tokens, instead of starting cold.
After compaction	The stack report and memory are loaded again automatically.

Safety fixes

- The secret-file guard no longer blocks harmless wildcards such as `ls src/*`, or a task description that only *mentions* `.env`. Hidden-file patterns (`.e*`), reading key files, and wildcards used with `find`, `rg` or `grep` are still blocked.
- The session report counts tasks in the folder actually in use.

Earlier releases

The tables below cover the earlier releases, up to `v2.1.0`.

Also in 2.1

Review finding (v2)	What changed
N1 checkpoint could commit secrets	Only the task's own files; secret files block the checkpoint; batches use their own branch
N3 browser checks couldn't run	Kit Playwright config, proven with a real browser
N4 old problems blocked reformatting	Shell-change check blocks only newly introduced violations
N5 secrets guard misfired	`.env.example` readable; safe `.gitignore`/seed commands allowed
N6 undo ignored dependents	Undo refuses, or `--cascade`
N7, N11 verifier unmeasured, skip rule vague	Verdicts recorded; first-pass rate in `stats`; skipping is a fixed rule
N8 Next 16 unchecked	Rewritten from the official docs, with sources
N9 no version control	Kit in git (tag `v2.1.0`), CHANGELOG, CI
N10 thin eval	Behaviour checks against the running app, hidden tests (incl. authorization), an ambiguous task, multi-run averages, a Next 16 variant; `--reference` proves the checks
N12–N15	Truncation warning, edit-hook timing, first-class global install, in-session batch nudge

Measuring the workflow

```
bash evals/run.sh --reference      # free: proves every check can pass (must be 5/5)
bash evals/run.sh                  # 3 real agent runs on Next 15 (costs API money)
bash evals/run.sh --next 16        # the Next 16 + cacheComponents variant
```

Results land in `evals/results/`. Compare them before and after changing any command, agent or skill.

Also in 2.0

Every finding from the architecture review is fixed and covered by the self-test (189 checks):

Review finding	What changed
H1 broad read permission	Removed `Read(//home/**)`; `~/.ssh` and `~/.aws` are now denied
H2 shell injection from package.json	Detected values are sanitised; a tampered cache file is ignored
H3 batches share one context	`run-tasks.sh` starts a fresh session per task; in-session batches are capped at 3
H4 no rollback	`--checkpoint` makes one commit per task; `tasks.sh undo`
H5 builder grades itself	`next-verifier` agent plus tagged criteria; browser checks via Playwright
M1 false-positive block	Network-call detection narrowed; `params.get()` is no longer blocked
M2 old type errors	Baseline: only new errors fail
M3 Next 16	New `next16.md` reference, version-aware session report, Next 16 advisories
M4 duplicate truth	`CLAUDE.md` holds a pointer only
M5 shell writes skip checks	`bash-guard` checks them
M6 no behaviour tests	`evals/run.sh`: fixture app, 4 tasks, objective checks, cost and score
M7 no numbers	`metrics.jsonl` + `tasks.sh stats`; the runner records cost per task
L1–L6	Read-only agents tightened, prompts de-duplicated, one hook dispatcher, `.env` shell guard, version stamp + uninstall, draft → approve gate